

Architecture over Parameters: How Far Agentic Design Patterns Carry a Weak Model in Automated Machine Learning

Stanislav Chumakov^a, Nikolay O. Nikitin^a

^a*ITMO University, Saint Petersburg, Russia*

Abstract

Large language models have made automated data science agents practical, but the field still lacks a measurement of what the surrounding architecture is worth. Reported gains routinely confound the choice of backbone model with the choice of harness, and agentic systems are seldom compared against baselines that a competent practitioner would tune. We hold the model fixed at a single open 31B-parameter model and vary the architecture across three systems that occupy distinct points of the design space: a rigid pipeline that configures a fixed AutoML backend, a six-agent multi-agent system, and an open agentic harness. All are evaluated inside a common execution environment on the official temporal splits of a real-world tabular benchmark, against eighteen expert-tuned reference methods.

Three findings follow. First, architecture is worth more than half of the achievable range: normalised against the published baselines, the systems span 0.26 to 0.84 under an identical backbone, yet none matches a tuned gradient boosting model at 0.87 across the eight tasks. Second, the relationship is not monotonic in architectural weight. The rigid pipeline ranks at the bottom of the leaderboard, the multi-agent system ranks highest, and the open harness falls between them; because the systems are compared as deployed, the runs that would separate an architectural effect from a prescriptive one are still outstanding. Third, the value of prescriptive domain knowledge changes sign with the starting point: it helps where a weak model would otherwise reach for an unsuitable primitive, and hurts where a working baseline already encodes the right choice. We further show that what the literature treats as one “knowledge injection” intervention is at least two, prescribing which tool to reach for and prescribing how to train and validate; that the tool prescription is the smaller part of the layer it names; and that reading the traces of the unprescribed system explains the boundary directly, since that system selects the prescribed family of tool unaided on every tabular task and never searches its hyperparameters. This offers an account of why aggregate ablations of knowledge injection have reported inconsistent effects. **[TODO: final numbers once the model axis lands]**

Keywords: automated machine learning, LLM agents, agentic design patterns, tabular learning, agent scaffolding, evaluation

1. Introduction

Automated machine learning has been reshaped by large language models. Systems built on them plan experiments, write and repair code, read execution logs, and interpret their own results, capabilities that classical AutoML frameworks, which search a fixed space of pipelines, never had. The design space of such systems is now large and largely uncharted: implementations differ in how much autonomy they grant the model, how they inject knowledge about libraries and methods, and how tightly they constrain the sequence of actions.

What has not been measured is how much any of this is worth. Papers in this area typically introduce one system and report that it improves on a previous one, with the backbone model, the scaffold, and the evaluation protocol all changing at once. Two consequences follow. Comparisons across papers cannot separate the contribution of the architecture from that of the model, and, more uncomfortably, agentic systems are rarely measured against the baseline a competent practitioner would reach for. Where a tuned gradient boosting model appears at all, it usually appears untuned.

This paper isolates the architectural axis. We fix the backbone to a single open 31B-parameter model and vary only the surrounding system across three points of the design space:

- **FEDOT.LLM**, a rigid pipeline in which the model interprets the task and configures a fixed AutoML backend, with code generated from templates that carry fixed input/output contracts;
- **AutoDS-Tools**, a multi-agent system of six specialised agents communicating through structured reports, with a shared toolkit and broad freedom of action;
- **Terminus-2**, an open harness that gives the model a shell, a working directory and an execution loop, and prescribes nothing else.

All three are executed inside a common environment that records every attempt, error, token and unit of cost, and all are evaluated on the official temporal splits of TabReD [1], a benchmark of real-world tabular tasks with distribution shift, against the eighteen expert-tuned reference methods published with it. Because those references are Optuna-tuned and averaged over fifteen seeds, they constitute a ceiling that reflects competent practice rather than a default configuration.

Contributions.

1. **A measurement of the ceiling.** Under an identical backbone, the architectures span 0.26 to 0.84 of the range defined by the published baselines, while a tuned gradient boosting model sits at 0.87 and the best published method at 0.95. No architecture we tested closes the remaining gap across the eight tasks, though the strongest comes within 0.03 of the tuned reference. We state this as a boundary rather than a defeat: it delimits what scaffolding can and cannot buy at a given model scale.
2. **A non-monotonic relationship, and a mechanism for it.** Performance does not increase with architectural weight, and it does not decrease with it either. The most constrained system ranks near the bottom of the leaderboard, and the highest-ranking of the three is neither extreme. We argue that constraint pays only where it removes a decision the weak model reliably gets wrong, and costs elsewhere, and we give direct evidence from an ablation whose sign flips between task families. The shape of the ordering, as opposed to its non-monotonicity, rests on runs that separate architecture from prescription and provisioning; Section 6 states what those are. **[TODO: decide the framing once the separating arms land: the inverted-U claim, or the weaker non-monotonicity claim that holds either way]**
3. **A separation of two axes usually conflated.** What the literature treats as one “knowledge injection” intervention is at least two: prescribing which class of tool to use (K_{tool}) and prescribing how to train and validate (K_{disc}). We show by direct measurement of the instruction layer that the tool prescription is between 8.6% and 25.4% of the text it gives its name to, that the two halves move the same task in opposite directions, and, by reading the traces of the system that receives no prescription, that on tabular data the tool half prescribes what the model already chooses unaided. Together these offer an explanation for the inconsistent effects reported for aggregate knowledge and memory ablations.

Relation to the neutral-scaffold assumption.. Terminal-Bench 2.0 introduces Terminus 2 as a deliberately minimal scaffold, a single tool and a headless terminal, built to mitigate the biases of richer agent scaffolds so that a leaderboard over it reads as a comparison between *models* [2]. That reading is only sound if the scaffold contributes little. It is already under pressure in the coding domain, where holding the model fixed and swapping the harness moves aggregate score by more than twenty points [3] and tokens per solved task by up to a factor of forty [4]. We ask the question where a strong non-agentic reference exists and can calibrate the answer: with the backbone held constant, architecture alone moves the normalised score by 0.58 of the range spanned by eighteen expert-tuned baselines. A scaffold is never neutral; on this domain it is the dominant term of the two. **[PENDING RUN: the model-axis runs that put a number on the model term for comparison]**

2. Related work

2.1. Degrees of autonomy in AutoML systems

Existing solutions can be ordered by how much of the modelling decision they leave to the system. At the lowest level sit automated hyperparameter optimisers, where recent work integrates language models mainly to address the

cold-start problem. Above them, pipeline composition searches jointly over feature engineering, model selection and hyperparameters. At the highest level, adaptive planning systems revise their search plan in response to intermediate results, expanding or contracting the space and reallocating budget as validation feedback arrives. Many published systems fall between levels rather than on them.

2.2. Knowledge integration

Orthogonally, systems differ in where their knowledge of libraries and methods comes from. Some rely entirely on the parametric memory of the backbone. Others retrieve auxiliary material to guide code generation, for instance strategic insights mined from competition write-ups [5]. A third group embeds API-level knowledge directly in the decision process, querying signatures and usage patterns at run time; MLZero, for example, combines semantic and episodic memory specifically to suppress hallucinated and outdated API calls [6].

We revisit this taxonomy in Section 4, where we argue that it conflates two interventions with different mechanisms.

2.3. Agentic systems for data science

Multi-agent frameworks for competition-style data science are now numerous [7, 8, 5, 6]. Their reported ablations disagree in an instructive way. On MLE-bench, search structure and exploration are reported as the largest single contributor in one system and as having no measurable effect in another, with memory ranked last in one study and level with search in a second [9]. Such disagreement is expected when ablations are run one factor at a time on different backbones and different task sets. The factors are also shown to interact directly: advanced search policies gain nothing when restricted to a weaker operator set [9].

2.4. Evaluation against tuned baselines

Agentic AutoML papers that do compare against classical methods almost always compare against defaults. The closest work to ours in domain is GRACE-DS [10], which evaluates LLM-powered AutoML agents on tabular tasks under a structured environment and reports a “capability equalizer” effect: structuring the interaction compresses the spread of end-to-end quality across backbones. Two differences matter here. First, its normalised scale is anchored on an *untuned* HistGradientBoosting oracle, so its unit of “solved” sits well below the tuned references we use. Second, its eight backbones are frontier and mixture-of-experts models in the 80B–1.6T range; no dense small model is evaluated. Its own results point the same way ours do: the largest gains from structure accrue to the strongest backbones, and the smallest to the weakest.

2.5. Scaffolding and model capability

Whether scaffolding can substitute for model capability has been asked directly in the embodied setting, where reasoning, memory, reflection and reinforcement modules were composed in a controlled factorial design over a capability ladder of dense models, concluding that external scaffolding can compensate for weaker native long-horizon reasoning [11]. We take that design as methodological precedent and ask the same question where a strong non-agentic reference solution exists, which is precisely what supervised tabular learning provides and embodied benchmarks do not. The agentic coding setting has produced a parallel line of evidence that the harness is a large hidden variable rather than a neutral wrapper: across configurable harnesses evaluated identically over models and tasks, the aggregate score spans 23.8 points [3], and harness choice induces up to a fortyfold difference in tokens per solved task while the model-to-model pass-rate spread stays within eight points [4]. That literature varies the harness with the task family fixed; we vary the architectural weight of the system with the task family fixed *and* with a tuned non-agentic reference available on the same split, which is what turns a spread into a calibrated fraction of an achievable range. A third result bears directly on our own design choice: when a coding harness is evolved automatically, ablations localise the gain to tools, middleware and long-term memory rather than to the system prompt [12]. Our prescribed-knowledge axis is a prompt-level intervention, and Section 4 treats it as an object of measurement rather than as a design pattern for precisely this reason. Our own finding, that its two halves act in opposite directions and largely cancel, is what that localisation predicts.

[TODO: at 1.5 pages; the draft is 16 of 18 with the drafting macros already discounted, so this is the first place to cut if the model axis needs room]

Table 1: The three systems as points in the design space.

	FEDOT.LLM	AutoDS-Tools	Terminus-2
Tool choice	fixed by design	free	free
Role decomposition	none	six agents	none
Plan	fixed stages	generated	emergent
Execution loop	bounded retries	per-agent	open
Input modalities	tabular only	unrestricted	unrestricted

3. Three architectures as design points

We describe each system by the questions that matter for our argument: where a modelling decision is taken, who can override it, how many execution iterations are available, and what is fixed beyond the model’s reach.

3.1. FEDOT.LLM: a rigid pipeline

FEDOT.LLM pairs an LLM-driven agent with the FEDOT framework [13] for tabular tasks, in four stages. Data understanding extracts file paths, variable types, target labels, metrics and submission format from the query and the dataset. Configuration and code generation produce a task-matched FEDOT configuration and select a code template with fixed input/output contracts, leaving only the implementation to the model; reusable helper functions encapsulate correct API usage and suppress hallucinated calls. An execution and correction loop runs the code in a sandbox with stdout/stderr monitoring, patching and re-running on failure until success or an iteration cap. Reporting compiles preprocessing, architecture and metrics, and persists the script, the trained pipeline and the predictions.

The decisive property for this paper is that the choice of modelling tool is not a decision at all: it is the architecture. The system cannot elect to solve a task with anything other than FEDOT, and it accepts only tabular inputs.

3.2. AutoDS-Tools: a multi-agent system

AutoDS-Tools coordinates six specialised agents that communicate through structured reports, forming a cumulative knowledge chain: an Analyst performs exploratory analysis; a Researcher studies the target library and produces usage cookbooks; a Planner turns both reports into an execution plan and a specification; a Coder iteratively refines the solution, adjusts parameters and optimises the pipeline against execution results; a Debugger absorbs execution errors without disrupting the main workflow context; and a Presenter audits the final solution. All agents share a common toolkit. The chain is literal rather than metaphorical: each stage writes a Markdown report into the working directory and the next stage reads it, so a trial directory contains `analyst_report.md`, `researcher_report.md` and `planner_report.md` as intermediate artefacts.

Freedom of action is broad: the system may use any library available in the environment and may restructure its approach between iterations.

3.3. Terminus-2: an open harness

Terminus-2 provides a shell, a working directory and an execution loop, and prescribes nothing about how the task should be solved. There is no planner, no role decomposition and no template. The model reads the task statement, inspects the data, writes code, runs it, reads the output, and repeats until it emits a submission or exhausts its budget.

4. The prescribed-knowledge axis

Interventions that supply a system with knowledge it would otherwise have to derive are usually treated as one thing. In our material they are two, and they act in opposite directions.

K_{tool} — tool prescription. Which class of instrument to reach for: a dedicated tabular AutoML library for tabular data, a vision backbone library for images, a graph learning library for graphs, a transformer library for text. It removes the question “what does one normally use for this?”

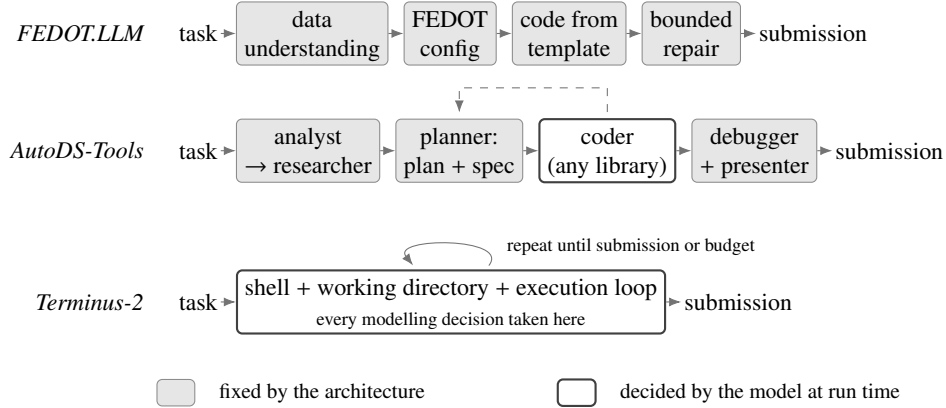


Figure 1: The three systems on one schematic. Shaded stages are fixed by the architecture and cannot be renegotiated by the backbone; unshaded stages are where the model actually decides what to do. The amount of prescribed structure falls from top to bottom, and—as Section 6 shows—measured performance rises in the same direction.

Table 2: Composition of the prescribed-knowledge layer, in characters. Only the first block varies with the task modality; the other two are fixed text appended to every task.

Family	LIBRARY	HARDWARE	DISCIPLINE	Tool share
tabular	1291	670	3122	25.4%
vision	569	670	3122	13.0%
nlp	369	670	3122	8.9%
graph	359	670	3122	8.6%

K_{disc} — **training discipline.** How to train and how to check oneself: the validation protocol, the handling of temporal shift, leakage checks, and the prohibition of degenerate submissions. It is orthogonal to the choice of library.

Separating them is not a matter of tidiness. In our runs the two halves move the same task in opposite directions. On `amp-parkinsons` the prohibition of constant submissions, a K_{disc} element added when that task was re-run, degraded SMAPE from 72.34 to 88.08, because a constant prediction scores well under SMAPE without modelling anything, whereas the library prescription is a K_{tool} element that acted independently. The pre-clause figure comes from the earlier re-run rather than from the published job, and is the one number in this paper we could not re-read from a run artefact. **[TODO: recover the pre-clause `amp-parkinsons` run, or drop the figure and make the point qualitatively]** While the two halves are delivered as a single instruction layer, no row of the ablation can be attributed to either, and the tables report the layer as a whole.

4.1. The layer is named after its smallest part

In the implementation we study the layer is composed literally as `instruction + LIBRARY[family] + HARDWARE + DISCIPLINE`, which makes the split measurable rather than conceptual. Table 2 gives the character counts of the version that produced the published runs.

Three observations follow. The tool prescription, the intervention the layer is named after in this paper and in our own earlier description of it, accounts for between 8.6% and 25.4% of the text. A fixed `HARDWARE` block of 670 characters, which pins down how the device is resolved before training, is neither tool prescription nor training discipline and forms a third category of its own. And the `DISCIPLINE` block, at 3122 characters, is the largest single component on every task.

This matters for attribution. Any effect ascribed to tool prescription in an aggregate ablation of this layer is at least as likely to originate in the two fixed blocks travelling with it, and the two blocks are constant across modalities while the reported effects are not. Separating the three is a change of a few lines at the composition site, and it is what the design in Section 5 requires.

Table 3: Position of the three systems on the prescribed-knowledge axis.

System	K_{tool}	K_{disc}	Left to the model
FEDOT.LLM	<i>always on, not removable</i>	switchable	configuration, template
AutoDS-Tools	switchable	switchable	everything
Terminus-2	switchable, off by default	switchable	everything

4.2. A prescription presupposes its environment

Tool prescription is realisable only where the prescribed tool exists, which makes it a property of the environment as much as of the text. The layer we study states that the specialized library is “pre-installed” and forbids falling back on general-purpose estimators, so it presumes a task image provisioned accordingly. Running the same text against an image without that library does not produce a weaker version of the treatment; it produces a failure, and the agent correctly reports that it could not comply.

We encountered this directly. The benchmark’s own task image ships `numpy`, `pandas` and `scikit-learn` on Python 3.13, and the prescribed tabular library does not build on that interpreter. An agent given the prescription in that image reads the data, writes a correct pipeline, and then fails at import, scoring zero while having done nothing wrong.

The consequence for the design is that the environment is part of the treatment and has to be stated alongside it. We therefore report the provisioning of each arm explicitly (Section 5) rather than treating the container as neutral background. It is also a caution for the literature: an ablation of “prescribe a specialized library” measures the prescription only where the provisioning already holds, and otherwise measures the provisioning.

4.3. Where each system sits

FEDOT.LLM is not missing K_{tool} : it has K_{tool} welded into its architecture. The instruction “solve this through FEDOT” is the pipeline. This is the extreme point of the factorial design rather than a hole in it, and it is what makes the result of Section 6 interesting: the system whose tool prescription is maximal and non-removable performs worst on exactly the task family the prescribed tool was built for.

The consequence for the experimental design is that only K_{disc} can be crossed with every system. K_{tool} is crossed on AutoDS-Tools and Terminus-2, with FEDOT.LLM entering as the reference point at which it cannot be switched off.

5. Experimental design

5.1. Factors

- **Architecture:** FEDOT.LLM, AutoDS-Tools, Terminus-2.
- K_{tool} : on/off, crossed on AutoDS-Tools and Terminus-2.
- K_{disc} : on/off, crossed on all three.
- **Backbone model:** `gemma-4-31b-it` throughout, plus a smaller and a larger model. **[PENDING RUN: model axis]**
- **Task protocol:** greenfield (nothing supplied) versus improve-a-baseline (a working training script supplied).

5.2. Benchmarks

TabReD. Eight real-world tabular tasks with temporal splits [1]. The benchmark ships no agentic protocol, so tasks are solved from scratch: this is our greenfield condition. Its value here is the published reference table of eighteen methods, spanning gradient boosting implementations, tabular deep learning architectures, ensembles and retrieval-augmented models, tuned with Optuna on the official validation split and averaged over fifteen seeds. Metrics are ROC-AUC for classification and RMSE in raw target units for regression.

MLAgentBench. Ten tasks spanning tabular, text, vision and graph data [14]. Each supplies a working but weak `train.py` and asks the agent to improve it: this is our improve-a-baseline condition. FEDOT.LLM participates only in the two tabular tasks, for the structural reason given in Section 9.

Scientific case studies. Three published studies with author-reported baselines, used in Section 10 to test external validity outside benchmark conditions. Their instructions carry the heaviest prescription of any condition in this paper: library, featurisation, leakage handling and the baseline to beat. They are read as the maximal-prescription end of the axis rather than as an unaided-agent condition.

5.3. Common execution environment

All runs execute inside the Harbor framework, which records for every trial the submission, the scorer output, the number of attempts and errors, wall-clock time, token counts and monetary cost. Reporting cost and attempts alongside accuracy lets us treat efficiency as a second axis rather than an afterthought.

5.4. Provisioning

Because a tool prescription presupposes the tool (Section 4), we state the container contents per arm rather than assuming a neutral environment. The Terminus-2 and FEDOT.LLM runs reported here used the benchmark’s stock image: `numpy`, `pandas` and `scikit-learn` on Python 3.13. The AutoDS-Tools arm also requires the libraries its prescription names, on Python 3.12, which is the environment its published runs also had. Data, splits, metrics and the backbone model are identical across arms; the installed library set is not, and we report it as a factor.

The same honesty is owed about the instruction. Each system enters the leaderboard in the configuration it is shipped and used in, which is not the same configuration for all three: Terminus-2 receives the task statement alone, FEDOT.LLM receives it through a pipeline whose tool choice is welded in, and AutoDS-Tools receives it with its prescribed-knowledge layer applied, that layer being the system’s default. A row of Table 5 therefore compares deployed systems rather than isolated architectures. Two further arms would separate the strands, AutoDS-Tools with the layer disabled on the common adapter and Terminus-2 on the enriched image, and each is a single sweep of the same eight tasks. We report which of them we ran. **[PENDING RUN: AutoDS-Tools without the layer, and Terminus-2 on the enriched image, on the common adapter]**

5.5. Noise floor

Rather than assume a significance threshold, we measure one. Terminus-2 and AutoDS-Tools were each run three times on every TabReD task; Table 4 reports the spread. The two do not share a floor. Terminus-2’s relative standard deviation ranges from 0.03% to 0.94%; AutoDS-Tools reproduces itself an order of magnitude more tightly, from 0.00% to 0.10%. On six of the eight tasks for Terminus-2 and three of the eight for AutoDS-Tools, two attempts are bit-identical, meaning the agent reproduced the same script, so the repeats carry less independent information than their count suggests.

We therefore treat relative differences above 1% as real when comparing the two systems, which takes the threshold from the noisier of the pair and is the conservative choice there. That threshold would be far too coarse for comparing configurations of one system against each other: for AutoDS-Tools the appropriate figure is set by its worst task, weather at 0.10%, not by its median of 0.01%. On MLAGentBench, where no such convergence is observed, we report three repeats per cell. **[PENDING RUN: MLAGentBench repeats]**

5.6. What is not crossed

Two limitations of the design are deliberate. FEDOT.LLM cannot enter the non-tabular cells at all. And the AutoDS-Tools TabReD runs reported in earlier drafts were computed on a different data slice; they are excluded from Table 5, which uses a re-run on the common adapter. Both sets are released, because the difference between them is itself the measured price of the data preparation.

Table 4: Run-to-run spread over independent attempts on the same task. A dagger marks a task on which two attempts agree to machine precision, i.e. the agent reproduced a bit-identical submission; this happens for 9 of the 16 task–system cells here, so repeats carry less independent information than their count suggests. The systems do not share a noise floor, which is why the threshold used for cross-system comparison is taken from the noisier of them.

Task	Metric	n	Mean	SD	Rel. SD
<i>Terminus-2</i>					
homesite-insurance	ROC-AUC	3	0.9588	0.00078	0.08%
ecom-offers [†]	ROC-AUC	3	0.5585	0.00419	0.75%
homecredit-default [†]	ROC-AUC	3	0.8590	0.00027	0.03%
sberbank-housing [†]	RMSE	3	0.2491	0.00234	0.94%
cooking-time [†]	RMSE	3	0.4837	0.00014	0.03%
delivery-eta [†]	RMSE	3	0.5482	0.00065	0.12%
maps-routing	RMSE	3	0.1628	0.00009	0.06%
weather [†]	RMSE	3	1.5247	0.00252	0.17%
<i>AutoDS-Tools</i>					
homesite-insurance	ROC-AUC	3	0.9599	0.00007	0.01%
ecom-offers	ROC-AUC	3	0.5800	0.00002	0.00%
homecredit-default	ROC-AUC	2	0.8627	0.00004	0.01%
sberbank-housing [†]	RMSE	3	0.2523	0.00015	0.06%
cooking-time [†]	RMSE	3	0.4819	0.00003	0.01%
delivery-eta [†]	RMSE	3	0.5474	0.00004	0.01%
maps-routing	RMSE	2	0.1620	0.00006	0.04%
weather	RMSE	3	1.4795	0.00149	0.10%

Two attempts bit-identical.

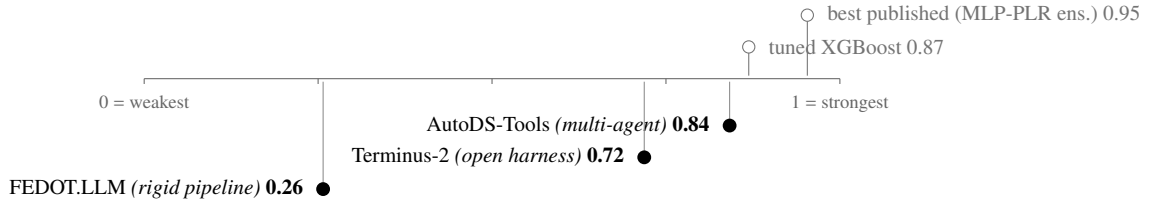


Figure 2: The three architectures (below the line) against the two marks that matter (above it), on the normalised scale where 0 is the weakest and 1 the strongest of the eighteen published baselines on each task. One backbone throughout.

6. Where architecture stops

Table 5 places the three systems that ran on the official TabReD splits into the benchmark’s own reference table, as if they were methods. Table 6 restates the same runs on a per-task normalised scale, where 0 is the weakest and 1 the strongest of the eighteen published baselines.

Three readings follow.

Architecture is worth about half the range.. With the backbone held fixed, the normalised mean moves from 0.26 for the rigid pipeline to 0.84 for the multi-agent system, a spread of 0.58 produced by the surrounding system, with the open harness at 0.72 in between. In average rank the same contrast is 18.3 versus 7.0 among twenty-one methods, with the open harness at 11.4.

But it does not reach competent practice.. A tuned gradient boosting model sits at 0.87 (rank 5.0) and the strongest published method at 0.95 (rank 4.1). The best of our systems reaches 0.84 at rank 7.0: close enough to sit inside the gradient-boosting band on individual tasks, not close enough to match it across the eight. The honest statement is therefore still a boundary, and now a narrower one: *with an open 31B model, none of the architectures we tested matches a competently tuned gradient boosting model across real-world tabular tasks under temporal shift, though*

Table 5: Agent architectures placed in the TabReD leaderboard. All rows are computed on the full official temporal splits with the benchmark’s own metrics in raw target units. Published baselines are Optuna-tuned means over 15 seeds [1]; the agent rows use a single backbone model (gemma-4-31b-it). Best per column in bold. A superscript on an average rank gives the number of tasks it was averaged over, where that is fewer than all eight.

Method	ROC-AUC $\uparrow$			RMSE $\downarrow$					Avg. rank
	Homesite	Ecom	HomeCred	Sberbank	Cooking	Delivery	Maps	Weather	
<i>GBDT</i>									
XGBoost	0.9601	0.5763	0.8670	0.2419	0.4823	0.5468	0.1616	1.4671	5.0
LightGBM	0.9603	0.5758	0.8664	0.2468	0.4826	0.5468	0.1618	1.4625	6.2
CatBoost	0.9606	0.5596	0.8621	0.2482	0.4823	0.5465	0.1619	1.4688	6.8
RandomForest	0.9570	0.5764	0.8269	0.2640	0.4884	0.5959	0.1653	1.5838	17.0
Linear	0.9290	0.5665	0.8168	0.2509	0.4882	0.5579	0.1709	1.7679	18.6
<i>Tabular DL</i>									
MLP	0.9500	0.6015	0.8545	0.2508	0.4820	0.5504	0.1622	1.5470	9.5
SNN	0.9492	0.5996	0.8551	0.2858	0.4838	0.5544	0.1651	1.5649	15.2
DCNv2	0.9392	0.5955	0.8466	0.2770	0.4842	0.5532	0.1672	1.5782	16.5
ResNet	0.9469	0.5998	0.8493	0.2743	0.4825	0.5527	0.1625	1.5021	11.6
FT-Transformer	0.9622	0.5775	0.8571	0.2440	0.4820	0.5542	0.1625	1.5104	8.9
MLP (PLR)	0.9621	0.5957	0.8568	0.2438	0.4812	0.5527	0.1616	1.5177	6.6
Trompt	0.9588	0.5803	0.8355	0.2509	0.4809	0.5519	0.1624	1.5187	10.5
<i>Ensembles</i>									
MLP ens.	0.9503	0.6019	0.8557	0.2447	0.4815	0.5494	0.1620	1.5186	7.1
MLP-PLR ens.	0.9629	0.5981	0.8585	0.2381	0.4806	0.5518	0.1612	1.4953	4.1
<i>Training methodologies</i>									
MLP aug.	0.9523	0.6011	0.8449	0.2659	0.4832	0.5532	0.1631	1.5193	13.1
MLP aug. rec.	0.9531	0.5960	0.7453	0.2515	0.4834	0.5541	0.1636	1.5160	13.5
<i>Retrieval-augmented</i>									
TabR	0.9487	0.5943	0.8501	0.2820	0.4828	0.5514	0.1639	1.4666	12.6
ModernNCA	0.9514	0.5765	0.8531	0.2593	0.4825	0.5498	0.1625	1.5062	11.4
<i>Agentic AutoML systems (this work)</i>									
Terminus-2 (<i>open harness</i>)	0.9588	0.5585	0.8590	0.2491	0.4837	0.5482	0.1628	1.5247	11.4
AutoDS-Tools (<i>multi-agent</i>)	0.9599	0.5800	0.8627	0.2523	0.4819	0.5474	0.1620	1.4795	7.0
FEDOT.LLM (<i>rigid pipeline</i>)	0.9400	0.5600	0.7700	0.2800	0.4900	0.5500	0.1700	1.5900	18.2

the strongest of them comes within 0.03 of it. We state it plainly because the comparison is usually made against defaults, where the same systems would appear to win outright.

The ordering is not monotonic in either direction.. The system that ranks highest is neither the most nor the least constrained but the one in between. Read as it stands, the relationship is an inverted U: the rigid pipeline at 0.26, the multi-agent system at 0.84, the open harness at 0.72. We state this with a caveat that the design section makes explicit and on which the reading depends. Each system enters in the configuration it ships in, and those configurations differ in more than architecture: the multi-agent row carries its prescribed-knowledge layer and an image provisioned with the libraries that layer names, while the open harness receives the task statement alone in the stock image. Until the two separating arms are run, one giving the multi-agent system with its layer disabled, and the open harness on the enriched image, “the middle-weight architecture wins” and “prescription plus provisioning wins” are not distinguishable by this table. **[PENDING RUN: both separating arms; the shape of the ordering is provisional until they land]**

The rigid pipeline sits at the bottom of the leaderboard.. FEDOT.LLM’s normalised mean is 0.26 and its average rank 18.3 of 21, placing it in the neighbourhood of plain linear regression (0.29 and 18.6). Its metrics reached us rounded to two decimals, so we checked how much of this placement is an artefact of that rounding: sweeping every task independently to the most and least favourable end of its rounding interval moves the normalised mean between 0.05 and 0.47 and the average rank between 20.3 and 16.6. The ordering against linear regression is therefore *not*

Table 6: Normalised position on each TabReD task: 0 is the weakest and 1 the strongest of the 18 published baselines. The two agent rows share one backbone model, so the spread between them is attributable to architecture alone.

Method	Homesite	Ecom	HomeCred	Sberbank	Cooking	Delivery	Maps	Weather	Mean
Terminus-2	0.88	-0.03	0.93	0.77	0.60	0.97	0.83	0.80	0.72
AutoDS-Tools	0.91	0.48	0.96	0.70	0.83	0.98	0.91	0.94	0.84
FEDOT.LLM	0.32	0.01	0.20	0.12	-0.21	0.93	0.09	0.58	0.26
XGBoost	0.92	0.39	1.00	0.92	0.78	0.99	0.96	0.98	0.87
MLP-PLR ens.	1.00	0.91	0.93	1.00	1.00	0.89	1.00	0.89	0.95
Linear	0.00	0.16	0.59	0.73	0.03	0.77	-0.00	-0.00	0.28

Table 7: Cost and wall-clock per trial, from the Harbor job telemetry. Wall-clock is measured end to end for the whole job and divided by the number of trials and by the job’s own concurrency setting, so it includes environment builds and any queueing; the two AutoDS-Tools rows cover the same eighteen tasks and differ only in the instruction file. Token prices are those of the shared backbone.

System	Benchmark	Trials	\$/trial	In tok/trial	Out tok/trial	Min/trial
Terminus-2	TabReD	24	0.0069	49 969	2 111	5.1
Terminus-2	MLAgentBench	6	0.0148	140 094	4 571	11.7
AutoDS-Tools (<i>K</i> on)	both	18	0.0174	140 292	10 017	130.1
AutoDS-Tools (<i>K</i> off)	both	18	0.0122	94 987	7 918	100.2
AutoDS-Tools (this work)	TabReD	24	0.0102	88 535	6 433	38.0

resolvable at the precision available to us, and we do not rest anything on it. What survives the whole interval is the part we do use: even at its most favourable corner the rigid pipeline stays 0.25 of the normalised range below the open harness, 0.37 below the multi-agent system, and below every tuned gradient-boosting baseline. We report this about one of our own systems deliberately: it is the lower anchor of the architectural axis, and its position is what gives the axis its length. **[PENDING RUN: full-precision FEDOT.LLM metrics from the Harbor job artefacts would collapse this interval; the reported point estimates already come from those runs, only at two decimals]**

Cost. The spread in accuracy is not bought with a matching spread in token cost. Table 7 reports the Harbor telemetry for the four jobs behind the results above.

Three things follow. First, token spend is not the binding constraint at this scale: a whole TabReD trial costs the open harness under a cent, and the most expensive row in the table is under two. What separates the systems from the tuned baselines is training compute and search, not the price of the backbone. Second, the ordering of accuracy and the ordering of cost do not agree: the system that ranks highest is also the cheapest per trial, by a factor of about 2.5 in tokens and an order of magnitude in wall-clock. Third, and most directly useful, the two AutoDS-Tools rows isolate the price of the prescribed-knowledge layer, since they differ only in the instruction file: adding it raises input tokens by 48%, output tokens by 26%, and dollars by 43%. Section 7 shows that what this buys is positive on some task families and negative on others; the cost, by contrast, is paid on every task.

The architecture runs out of time before it runs out of money. The mean in that table hides the shape of the distribution, and the shape is the finding. In our own re-run, twenty-four trials under the benchmark’s own 3600 s per-task ceiling, agent wall-clock ranged from 17 to 60 minutes. Five trials finished within fifteen minutes of the ceiling and two reached it and were killed, producing no submission at all. The open harness runs the same eight tasks in about five minutes each under the same ceiling. Two lost trials out of twenty-three is an 8.7% loss rate caused by nothing but elapsed time, in a design that has only three attempts per cell, and it concentrates on the slowest tasks, so it is not random with respect to the result either. We report the two as failures rather than as zeros, since a timeout is a property of the budget and not a measurement of the model (Section 11). The practical reading is that at this scale token cost is not the binding constraint and wall-clock is: an architecture that spends fifty minutes where another spends five is buying its accuracy partly with a margin it does not have.

The wall-clock column should be read with care. It is end-to-end job time divided by trials and by the job’s own concurrency, so it absorbs environment builds and any queueing, and the jobs were not run on a common machine. It

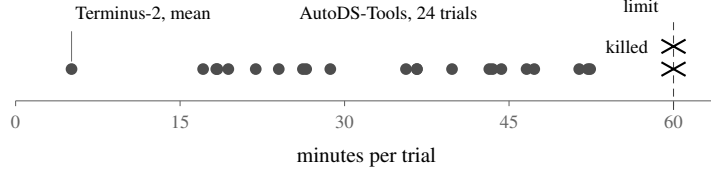


Figure 3: Agent wall-clock per trial on TabReD. Each dot is one AutoDS-Tools trial; the two crosses are the trials that reached the task’s own limit and were killed before writing a submission. The open harness averages five minutes on the same tasks under the same limit.

Table 8: Effect of the prescribed-knowledge layer on AutoDS-Tools over MLAGentBench, the improve-a-baseline condition. Both branches run the identical six-agent architecture and differ only in the instruction file. The layer is delivered as one block, so no row can be attributed to K_{tool} or K_{disc} separately (Section 4). Tasks are ordered by effect size; the sign of the effect flips between the top and bottom.

Task	Metric	layer on	off	Δ (norm.)
identify-contrails	Dice	0.3257	0.0006	+1.00
ogbn-arxiv	Accuracy	0.5560	0.3217	+0.42
feedback	MCRMSE	0.5244	1.6634	+0.68
amp-parkinsons	SMAPE	88.0821	96.5844	+0.09
spaceship-titanic	Accuracy	0.8085	0.8103	-0.00
house-price	MAE	15 956.8	15 671.5	-0.02
imdb	Accuracy	0.8397	0.8643	-0.03
cifar10	Accuracy	0.8228	0.8849	-0.07
clrs	Ptr. acc.	0.0525	0.4298	-0.88
fathomnet	micro-F1	0.0300	0.1676	-0.82

is reliable as an order of magnitude and as a within-pair comparison, since the two AutoDS-Tools branches ran on the same setup two days apart, and not as a benchmark of throughput.

7. Why more structure is not better

The ordering in Section 6 does not follow architectural weight. The most constrained system performs worst, and the best of the three sits between the extremes rather than at either end. A naive reading of the hypothesis that design patterns compensate for weak models is therefore not supported: quantity of structure does not predict outcome.

We propose a sharper statement, which the data can test:

Constraint pays exactly where it removes a decision the weak model reliably gets wrong. Everywhere else it removes exploration the model could have recovered from.

Table 8 gives direct evidence. It reports the ablation of the prescribed-knowledge layer on AutoDS-Tools over MLAGentBench, where both branches run the identical six-agent architecture and differ only in the instruction file. The layer arrives as one block, so what the table isolates is the layer, not K_{tool} alone (Section 4); we read the sign flip below as evidence about override in general, and the runs that separate the two halves are the design of Section 5.

The sign of the effect is not random across tasks; it separates them into two groups. Where the weak model would otherwise reach for an unsuitable primitive, prescription produces large gains: Dice rises from 0.0006 to 0.3257 on *identify-contrails*, MCRMSE falls from 1.6634 to 0.5244 on *feedback*, and accuracy rises from 0.3217 to 0.5560 on *ogbn-arxiv*. Where the supplied training script already encoded a suitable choice, the same prescription overrides it and does damage: micro-F1 falls from 0.1676 to 0.0300 on *fathomnet* and pointer accuracy from 0.4298 to 0.0525 on *clrs*.

This is the mechanism seen from both sides. Prescription is an override; it is valuable when the default it overrides is wrong and costly when the default is right. FEDOT.LLM is the limiting case, permanently in override mode, which is consistent with its position at the bottom of Table 5.

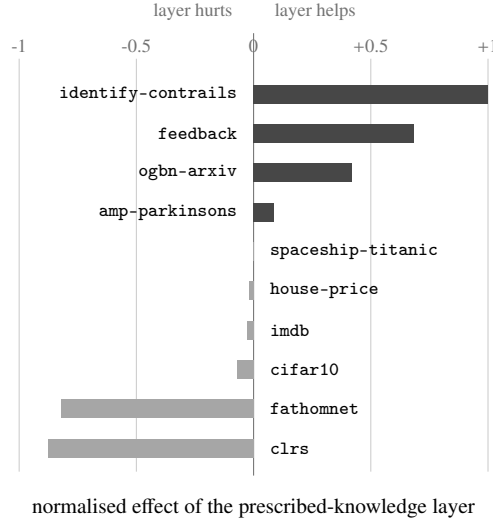


Figure 4: The same instruction layer, the same architecture, the same backbone: only the task family changes. The effect runs from +1.00 to -0.88, so an aggregate over these ten tasks would report close to nothing.

7.1. What the unprescribed system chooses on its own

The mechanism makes a prediction that can be checked without an ablation: if prescription only pays where the model’s default is wrong, then on a task family where the default is already right it should be worth close to nothing. We tested this by reading the terminal traces of the open harness, the system that receives no prescription of any kind, and recording which learner it reached for unaided.

The instruction it receives names no library: it states the task, the file layout and the required output format, and says to train and validate any model of the agent’s choosing. On all eight TabReD tasks it fitted a gradient-boosting model regardless: LightGBM on seven, and on the eighth XGBoost together with scikit-learn’s histogram-based gradient boosting. On six of the eight it first installed LightGBM into the stock image itself. This is the same family the tool prescription would have prescribed. On the tabular family the prescription is therefore not an override at all but a restatement of the default, which is why we expect, and find, no room for it to help.

The same traces explain the ceiling of Section 6 sharply. The configuration is hand-written and near-identical across tasks: learning rate 0.05 with 1000 estimators on seven of the eight, early stopping on a holdout on seven, 31 leaves wherever LightGBM’s leaf count is set at all. No search of any kind appears in any of the eight traces: no Optuna, no grid, no randomised search, no cross-validated sweep. The gap to the published gradient-boosting rows is thus not a gap in tool selection, which the harness gets right unaided, but a gap in search: the baselines are Optuna-tuned and averaged over fifteen seeds, the agent writes one plausible configuration and stops. Read together with Table 8, this locates the useful intervention precisely. Telling a weak model *which* tool to use is worth a great deal on task families where it would otherwise choose badly and nothing on tabular data; what is left unaddressed on tabular data, search over the chosen tool, is the part no system in our comparison performs. The per-task extraction behind this paragraph is released with the paper.

7.2. Execution friction as a mediator

The trace telemetry shows the mechanism operating rather than merely its result. AutoDS-Tools contains a repair loop that is invoked when execution fails; it is part of the system and is active in both branches, but it does not fire equally often. Across the 36 runs it was invoked 18 times with tool prescription against 12 without, in 9 of 18 runs against 8 of 18. The imbalance concentrates exactly where the prescription pushes hardest: 4 invocations against 0 on `clrs` and 3 against 0 on `amp-parkinsons`, both cases where the agent is steered to a specialised library whose API it then fails on. The reverse also occurs, at smaller magnitude: 0 against 2 on `spaceship-titanic` and 3 against 4 on `imdb`.

Table 9: The same layer ablation in the greenfield condition. Nothing is supplied beyond the task statement, and the branch carrying the prescription wins 6 of 8—the opposite of the improve-a-baseline count in Table 8. These runs used an earlier data preparation and are not comparable to the leaderboard; both branches share it, so the contrast is internally valid.

Task	Metric	layer on	off	Δ (norm.)
homesite-insurance	ROC-AUC	0.9616	0.9591	+0.003
ecom-offers	ROC-AUC	0.5837	0.6197	-0.058
homecredit-default	ROC-AUC	0.8641	0.7621	+0.118
sberbank-housing	RMSE	0.2561	0.2473	-0.034
cooking-time	RMSE	0.4864	0.4878	+0.003
delivery-eta	RMSE	0.5639	0.5656	+0.003
maps-routing	RMSE	0.1657	0.1666	+0.005
weather	RMSE	1.5258	1.5611	+0.023

Read as a mediator rather than as noise, this quantifies the cost side of the mechanism. Prescription raises execution friction, and its net effect on the score is positive only where the prescribed tool is enough better to repay that friction. It also means the effect we measure is that of prescription *as delivered by a system able to recover from the errors prescription causes*; in a harness without a repair loop the losses would plausibly be larger. Whether prescription still pays without repair is a crossed question of its own, and a cheap one; see Section 11.

It also offers an explanation for the disagreement in the literature noted in Section 2. If “knowledge injection” bundles an override that helps with one that hurts, and the mix of task families differs between studies, then aggregate ablations will report inconsistent and sometimes null effects for reasons that have nothing to do with the systems being compared.

[TODO: extend the trace reading of Section 7.1 to each large-loss row of Table 8]

8. The starting point changes the sign

The two benchmarks differ in a way that is usually treated as a nuisance to be disclosed. TabReD supplies nothing and the task is solved from scratch; MLAGentBench supplies a working training script and asks for improvement. We treat this as a factor rather than a caveat, because the value of prescriptive structure depends on it.

The win–loss counts split cleanly along that line. In the greenfield condition, the eight TabReD tasks where nothing is supplied, the branch carrying the prescription wins six of eight. In the improve-a-baseline condition, the ten MLAGentBench tasks that each ship a working `train.py`, the branch without it wins six of ten. Both counts come from the same system, the same backbone and the same pair of instruction files; only the starting point differs. The reading is the one the mechanism predicts: where there is no default to override, any prescription that points the model at a reasonable tool improves on what it would otherwise construct, and where the supplied script already contains the decision the prescription was introduced to make, overriding it discards work that was correct.

The counts are symmetric but the magnitudes are not, and that asymmetry is the more informative half. In the improve-a-baseline condition the normalised effect of the prescription runs to ± 1.00 (Table 8); in the greenfield condition it is an order of magnitude smaller, a median of 0.014, with only `homecredit-default` exceeding 0.06. Every one of the eight effects still clears the run-to-run spread measured in Section 5, whose median on these tasks is 0.002, so they are real rather than noise. But on tabular data the prescription is doing very little work in either direction. Section 7.1 explains why: the system reaches for the prescribed family unaided, so there is almost nothing left for the prescription to change. Greenfield does not mean structure pays richly; it means structure stops costing.

The TabReD half of this count carries a caveat of its own. Those eight runs used an earlier data preparation and are not comparable to the leaderboard of Section 6 (Section 11); they are, however, an internally valid contrast, because both branches ran on exactly the same preparation. The noise floor they are compared against was measured on the official splits and on a different system, so it is indicative rather than matched.

This interaction is what the two protocols measure jointly, and it is why the aggregate win–loss count across the two benchmarks should not be read as a single comparison.

[TODO: restate as a 2x2 once the K-disc grid is complete; the current evidence is one factor observed across two protocols, not a crossed design]

Table 10: Coverage and the reason for each absence.

System	TabReD	MLAgentBench	Reason for absences
FEDOT.LLM	8/8	2/10	tabular inputs only, by design
AutoDS-Tools	8/8	10/10	—
Terminus-2	8/8	6/10	four not attempted; no failures

9. Limits of applicability

Coverage across the three systems is uneven, and the unevenness is a result rather than a gap. FEDOT.LLM attempts two of the ten MLAgentBench tasks not because runs were abandoned but because its architecture accepts tabular inputs only; the segmentation, multi-label vision, graph and sequence tasks are outside what it can express, and would remain so after substantial engineering. Its domain is therefore the tabular core: the eight TabReD tasks plus house-price and spaceship-titanic.

We consequently report two things separately. Head-to-head comparisons are made on the tabular core, where all systems ran. Tasks outside it are reported as a coverage analysis, with an explicit reason for each absence, whether structurally unsupported, run failed, or not attempted, rather than as blank cells.

Terminus-2 is missing four MLAgentBench tasks, `identify-contrails`, `feedback`, `clrs` and `fathomnet`, for a different and much weaker reason: the job that produced its MLAgentBench numbers contains six trials, all completed with no errors, and the remaining four were never submitted. Nothing structural prevents them, and we mark them as not attempted rather than as unsupported. [\[PENDING RUN: run the four missing tasks\]](#)

Read this way, the coverage table is itself a finding: the architecture that performs worst on the tasks it can attempt is also the one that can attempt the fewest, and both properties follow from the same design decision.

10. Scientific case studies: the far end of the axis

Benchmarks measure what they were built to measure. To see the behaviour above outside benchmark conditions, we ran AutoDS-Tools on three published scientific studies, each with an author-reported baseline on the authors’ own split.

These three runs are not a knowledge-free condition, and it matters for reading them that they are the opposite. Their instructions sit at the far end of the prescribed-knowledge axis of Section 4: each names the library to use under a heading marked `REQUIRED`, supplies a working code snippet that constructs it, identifies the target-leaking columns and confirms they have been removed, describes the split design, and states the author baseline as the number to beat. The polymer instruction goes further and restates the published finding itself, that Morgan fingerprints with gradient boosting is the strongest model for this property, together with the fingerprint radius and bit width. What these runs measure is therefore the ceiling under *maximal and correct* prescription, which is a useful quantity precisely because Section 7 shows prescription is not always correct.

Maize yield prediction.. Grain yield for field plots from the Genomes-to-Fields initiative, on the authors’ own split with no environment leakage. The agent reached Pearson $r = 0.569$ against the authors’ best model at $r = 0.461$ and their neural network at $r = 0.426$. The correlation is higher; the error is not. The agent’s normalised RMSE is 0.958 against the authors’ 0.948, and its R^2 on the held-out set is 0.083. It therefore ranks the plots better than the published model while predicting the yield itself no more accurately, a distinction that matters for the applied question and that a single-metric comparison would hide.

F-DATA exit-state prediction.. Job success or failure on Fugaku telemetry, using only attributes known at submission time, with the temporal split taken from the paper. The agent reached 92.07% accuracy against an author baseline of 89%. The instruction states the relevant context and the verifier confirms it: the classes are imbalanced and a constant “always success” predictor already scores 88.08%. Measured against that floor rather than against zero, the authors gain 0.9 points and the agent 4.0. Balanced accuracy is 0.705, so most of the remaining error is on the minority class, which is the class an operator would want predicted.

OpenPoly glass transition temperature.. Polymer property regression from PSMILES strings. The agent reached $R^2 = 0.543$ against an author baseline of 0.904, with MAE 40.4 K against 14.6 K. This comparison is *not* like-for-like: the authors used the full dataset, we were unable to obtain it, and the agent worked from a subset whose test split is 89 rows. We report the row for completeness and draw no conclusion from it.

What this does and does not show.. Two of three author baselines were exceeded on the metric the authors reported, by a 31B open model. That is a real result about applied scientific baselines, which are frequently well below what a competent tabular pipeline achieves, and it supports reading the ceiling of Section 6 as a statement about the gap to *tuned* tabular baselines specifically rather than about the systems being weak in general. It is not a result about agents working unaided: the knowledge that would otherwise have to be discovered, which library, which featurisation and which columns leak, was supplied in the prompt. Read against Section 7 the two findings agree. Prescription pays where the model’s own default would be wrong, and on an unfamiliar modality such as polymer SMILES the default would certainly be wrong; the tabular benchmark tasks, where the default is already right, are where the same prescription buys nothing.

[PENDING RUN: Terminus-2 and FEDOT.LLM on the same three cases, both with these instructions and with them stripped to the task statement, which would turn this section into a two-by-three design rather than three single points]

11. Threats to validity

Comparability of data slices.. The TabReD results in Table 5 are restricted to systems that ran through the adapter which uses the full official temporal splits and the benchmark’s own metric definitions in raw target units. Earlier AutoDS-Tools runs used a different preparation: a 40,000-row training slice spanning the official train and validation splits, and a 10,000-row test slice taken from the end of the official test window, with an additional row-order column exposed that the official protocol withholds. Rebuilding the tasks through the common adapter puts an exact figure on the difference: on cooking-time the official split is 227,087 train / 51,251 validation / 41,648 test (319,986 rows in total, matching the benchmark’s published instance count), so the earlier test slice covered 24.0% of the official test window. Rebuilding all eight tasks puts the earlier slice at 16.7%–61.4% of the official test window, in every case its final rows, i.e. the point of maximum temporal drift. sberbank-housing is the one exception: there the official test split is itself only 4,647 rows, and the earlier runs used all of them. Measured split sizes for all eight tasks are released with the paper. Those numbers are not leaderboard-comparable and are excluded here; Table 5 uses a re-run of the same system on the common adapter, and both sets are released so that the difference between them can be read as the price of the preparation.

Environment provisioning differs between arms.. Beyond the data slice, the published comparison also differed in what was installed. The AutoDS-Tools runs used images carrying the specialized libraries its prescription names; the Terminus-2 and FEDOT.LLM runs used the benchmark’s stock image. This is a second axis of non-comparability, and it is not removed by aligning the data. We report provisioning per arm and note that the direction of the residual advantage favours AutoDS-Tools, i.e. against our conclusion that the open harness ranks highest, so the reported ordering is conservative with respect to this difference. The traces bound how large the residual can be on the tabular family: on six of the eight TabReD tasks the open harness simply installed LightGBM into the stock image itself before using it, so a missing library delayed it rather than blocking it. That does not generalise to modalities whose libraries are heavier to install, and it is exactly where a matched arm would be informative. A fully matched arm, in which every system is offered the same enriched image, would remove the axis outright; we treat it as a contingency rather than a requirement, because it decides nothing unless AutoDS-Tools overtakes the open harness once provisioned.

Repeats.. Published baselines are means over fifteen seeds; our agent rows are one run for FEDOT.LLM and the mean of three attempts for Terminus-2 and AutoDS-Tools. The measured noise floor (Section 5) justifies this on TabReD but not elsewhere. The floor is not the same for the two systems: Terminus-2’s relative standard deviation reaches 0.94%, while AutoDS-Tools reproduces its own result to within 0.01%. The 1% threshold we apply to the cross-system comparison is therefore set by the noisier of the pair and is the conservative choice there; it would be far too coarse for comparing configurations of AutoDS-Tools against each other, where a tenth of a percent is already outside the spread.

Numeric precision.. FEDOT.LLM metrics were recorded to two decimal places, which is insufficient to separate it from neighbouring methods on tasks where the published spread is narrow. Sweeping every task independently to both ends of its rounding interval moves its average rank between 16.6 and 20.3 of 21 and its normalised mean between 0.05 and 0.47 (Section 6); no claim in this paper depends on any part of that interval, but the individual rank cells of Table 5 for that row should be read as indicative. [\[PENDING RUN: full precision from the run artefacts\]](#)

Degenerate submissions, and scorers that accept them.. A prohibition on constant submissions was present for two MLAGentBench tasks and absent for the other eight, having been added when those two were re-run. Since it constrains rather than assists, it acted against the branch that carried it. We inspected the submissions behind the two results this could distort, and both are affected. On `imdb` the open harness submitted a single label for all 25,000 test rows; the scorer returned an accuracy of exactly 0.5000 with no degeneracy check, so a submission containing no model at all is recorded at chance level rather than rejected. On `amp-parkinsons` the submission is not constant but is largely so: 344 of 559 rows (61.5%) carry the same global value in every one of the four targets, and only the remainder is fitted, which is enough to explain a SMAPE that would otherwise look surprisingly good. We therefore exclude the `imdb` figure from any claim and read the `amp-parkinsons` figure as a partial-fallback result rather than a model score. The wider point is a property of the benchmark rather than of any system in the comparison: on eight of the ten tasks nothing in the scoring path distinguishes a trained model from a constant, and an agent that fails silently is scored as if it had merely performed poorly.

An active repair loop inside the treated system.. The repair loop of AutoDS-Tools is part of the architecture and not part of the prescribed-knowledge intervention, but it is invoked asymmetrically between the branches (Section 7). It therefore partially masks the cost of prescription. We report the invocation counts alongside the scores rather than adjusting for them, and note that running both branches with the repair loop disabled would measure a different quantity, the interaction between prescription and repair, which we intend to run but is not the contrast reported here. [\[PENDING RUN: \$K_{\text{tool}} \times \text{repair-loop } 2 \times 2\$ \]](#)

Trials lost to the wall-clock ceiling.. Two of the twenty-four AutoDS-Tools trials on the common adapter reached the task’s 3600 s limit and were terminated before writing a submission, on `homecredit-default` and `maps-routing`. Both are excluded from the reported means, which therefore rest on two attempts rather than three for those two tasks. We considered and rejected the alternative of scoring them as zero: a timeout measures the budget, not the model, and averaging it in would have moved `homecredit-default` from 0.863 to 0.575 and dropped the system from third place in Table 5 to near-last, an artefact that would have looked entirely plausible. The exclusion is not neutral either: it drops the slowest attempts, so the reported means are mildly optimistic about a system whose failures correlate with the tasks it finds hardest. Raising the ceiling would settle this, and would also change what is being measured.

Subsampled data.. Several MLAGentBench tasks use subsets of the original data, so absolute values are not comparable with public leaderboards; none uses an official competition test split. The comparisons in this paper are internal by construction.

Validation access.. Agents running through the common TabReD adapter receive a labelled validation split, whereas the published baselines train on the training split alone. This is a small advantage to the agents and works against the direction of our main conclusion.

Single backbone family.. All results reported here use one model family. [\[PENDING RUN: model axis\]](#)

12. Conclusion

We isolated the architectural axis of agentic AutoML by holding the backbone model fixed and varying only the surrounding system, and we measured the result against reference methods that were tuned.

Architecture is worth a great deal and not quite enough. Under an identical open 31B model, the systems span 0.26 to 0.84 of the range defined by eighteen expert-tuned baselines, while a tuned gradient boosting model sits at 0.87. The gap that architecture closes is real and large; the gap it leaves open has narrowed to 0.03 but has not closed, and on the eight tasks taken together no configuration we tested matched the tuned reference.

The relationship is not monotonic in architectural weight, in either direction. The most constrained system ranks near the bottom of the leaderboard, and the highest-ranking of the three is neither extreme, which suggests that the useful question is not how much structure a system imposes but which decision that structure removes. Our evidence is that constraint pays where it overrides a default the weak model gets wrong, and costs where the default was already right. The mechanism is visible directly in an ablation whose sign flips between task families, and one that plausibly explains why aggregate knowledge and memory ablations in the literature disagree with each other.

For practitioners the implication is narrow and useful. On established task families with strong classical solutions, a tuned baseline remains the correct choice, and an agentic system’s value lies in coverage, in cost, and in reaching tasks where no such baseline exists, as the scientific case studies show, where two of three published author baselines were exceeded by an open 31B model that was told which library and which featurisation to use. What the practitioner supplies, and whether the system would have found it alone, is the variable to watch.

Data availability

All tasks, both instruction variants and every execution trace behind the numbers in this paper are public. The benchmark adaptations and the two instruction files per task are on HuggingFace as `danil-e/harbor-datasets-tabred` and `danil-e/harbor-datasets-mlab`; the tabular adapter used for the leaderboard-comparable runs is `AaLexUser/harbor-tabred-adapter`. The runs themselves, meaning trajectories, submissions, scorer outputs and per-job cost and token telemetry, are on the Harbor hub under job identifiers `7593cc7b` and `1a42d1fe` (Terminus-2 on TabReD and on MLAGentBench) and `3113aba7` and `56768b37` (the two AutoDS-Tools instruction branches). Any single trial can be retrieved with `harbor hub trial download`, which reproduces the trial directory including `verifier/reward.json`; the values in our tables were checked against those files task by task. Released alongside the paper are the consolidated metric table across all systems and benchmarks, the measured official split sizes for the eight TabReD tasks, the extraction of learner and hyperparameter choices from the Terminus-2 traces used in Section 7.1, the job telemetry of Table 7, and the scripts that turn them into every table printed here.

[TODO: repository DOI once deposited]

Acknowledgements

[TODO:]

References

- [1] I. Rubachev, N. Kartashev, Y. Gorishniy, A. Babenko, TabReD: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks (2024). `arXiv:2406.19380`.
URL <https://arxiv.org/abs/2406.19380>
- [2] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al., Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, in: International Conference on Learning Representations (ICLR), 2026, terminal-Bench 2.0; introduces the Terminus 2 scaffold and the Harbor harness. `arXiv:2601.11868`.
URL <https://arxiv.org/abs/2601.11868>
- [3] Y. Yao, X. Tan, C.-H. Liu, Y. Li, Z. Wang, W. Yu, Z. Tan, Y. Tian, G. Zhao, L. Sun, X. Zhang, T. Yang, Harness-Bench: Measuring harness effects across models in realistic agent workflows (2026). `arXiv:2605.27922`.
URL <https://arxiv.org/abs/2605.27922>
- [4] N. Vats, O. Golev, The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation (2026). `arXiv:2607.22585`.
URL <https://arxiv.org/abs/2607.22585>

- [5] S. Kulibaba, A. Dzhililov, R. Pakhomov, O. Svidchenko, A. Gasnikov, A. Shpilman, KompeteAI: Accelerated autonomous multi-agent system for end-to-end pipeline generation for machine learning problems (2025). arXiv:2508.10177.
URL <https://arxiv.org/abs/2508.10177>
- [6] H. Fang, et al., MLZero: A multi-agent system for end-to-end machine learning automation (2025). arXiv:2505.13941.
URL <https://arxiv.org/abs/2505.13941>
- [7] Z. Li, Q. Zang, D. Ma, J. Guo, T. Zheng, M. Liu, X. Niu, Y. Wang, J. Yang, J. Liu, W. Zhong, W. Zhou, W. Huang, G. Zhang, AutoKaggle: A multi-agent framework for autonomous data science competitions (2024). arXiv:2410.20424.
URL <https://arxiv.org/abs/2410.20424>
- [8] P. Trirat, W. Jeong, S. J. Hwang, AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML (2025). arXiv:2410.02958.
URL <https://arxiv.org/abs/2410.02958>
- [9] E. Toledo, K. Hambardzumyan, M. Josifoski, R. Hazra, N. Baldwin, A. Audran-Reiss, M. Kuchnik, D. Magka, M. Jiang, A. M. Lupidi, A. Lupu, R. Raileanu, K. Niu, T. Shavrina, J.-C. Gagnon-Audet, M. Shvartsman, S. Sodhani, A. H. Miller, A. Charnalia, D. Dunfield, C.-J. Wu, P. Stenetorp, N. Cancedda, J. N. Foerster, Y. Bachrach, AI research agents for machine learning: Search, exploration, and generalization in MLE-bench (2025). arXiv:2507.02554.
URL <https://arxiv.org/abs/2507.02554>
- [10] A. Tsymbalov, D. Zaripov, A. Epifanov, A. Palienko, GRACE-DS: a guarded reward-guided agent correction environment in data science, normalised scale anchored on an untuned HistGradientBoosting oracle; eight frontier/MoE backbones, no dense small model (2026). arXiv:2606.16000.
URL <https://arxiv.org/abs/2606.16000>
- [11] J. Chen, J. Shen, H. Kang, Z. Hong, Q. Jiang, S. Bose, Y. Zhang, L. Leng, A. Vyas, L. Mao, S. Ouyang, K. Zhou, L. Qin, AgentSpec: Understanding embodied agent scaffolds through controlled composition (2026). arXiv:2606.14674.
URL <https://arxiv.org/abs/2606.14674>
- [12] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, Y.-G. Jiang, Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses (2026). arXiv:2604.25850.
URL <https://arxiv.org/abs/2604.25850>
- [13] N. O. Nikitin, P. Vychuzhanin, M. Sarafanov, I. S. Polonskaia, I. Revin, I. V. Barabanova, G. Maximov, A. V. Kalyuzhnaya, A. Boukhanovsky, Automated evolutionary approach for the design of composite machine learning pipelines, *Future Generation Computer Systems* 127 (2022) 109–125. doi:10.1016/j.future.2021.08.022.
- [14] Q. Huang, J. Vora, P. Liang, J. Leskovec, MAgentBench: Evaluating language agents on machine learning experimentation (2024). arXiv:2310.03302.
URL <https://arxiv.org/abs/2310.03302>